TEXAS McCombs
The University of Texas at Austin
McCombs School of Business

Great Learning
POWER AHEAD

AI Agents and Generative AI for Business Applications

Single Agent System



Reimbursement Automation

Problem Statement

Business Context

In many mid-sized and large organizations, **employee expense processing and reimbursement** continues to rely heavily on manual, paper-based workflows. Employees submit physical or scanned receipts, which finance teams must individually review, categorize, and validate against company expense policies. This process demands significant time and effort to verify expense details such as category, amount, date, vendor, and compliance requirements.

As organizations grow, the **volume of expense claims increases substantially**, placing additional strain on finance teams. Manual processing creates operational bottlenecks, slows reimbursement cycles, and raises processing costs. It also increases the likelihood of human errors, duplicate reimbursements, and policy violations. Delays in reimbursements can negatively affect employee satisfaction and erode confidence in internal financial processes.

Additionally, expense policy interpretation often varies across reviewers, leading to **inconsistent approval decisions** and increased audit and compliance risks. The lack of automation limits real-time visibility into organizational spending patterns, making it difficult for finance teams to proactively enforce controls, detect anomalies, or identify potential misuse early in the process.

Objective

The objective is to simplify and streamline the expense processing and reimbursement workflow by leveraging intelligent automation. Routine tasks such as receipt data extraction, expense categorization, and policy validation can be handled automatically, reducing manual effort and processing time.

- Reduce manual effort in reviewing and processing expense claims
- Minimize errors and policy non-compliance in reimbursements
- Accelerate reimbursement turnaround time for employees

Data Description

This dataset is a structured reimbursement database containing employee, reimbursement history, and policy information.

Reimbursement Database Tables

1. **Employee Details Table** - Employee ID, Name, Position
2. **Past Reimbursements Table** - ID, Employee ID, Reimbursement Date, Category, Amount
3. **Reimbursement Policy Table** - Position, Category, Reimbursement Amount (max allowable)

Solution Approach

The notebook implements a single agent using LangGraph's prebuilt `create_agent`, where a single intelligent agent processes an employee reimbursement request by interleaving reasoning with specialized tool calls instead of following a hardcoded execution pipeline. Rather than embedding business logic directly inside the language model, the agent dynamically decides which tool to invoke next, observes the result, and continues reasoning until the reimbursement request has been completely processed.

Architecture Components

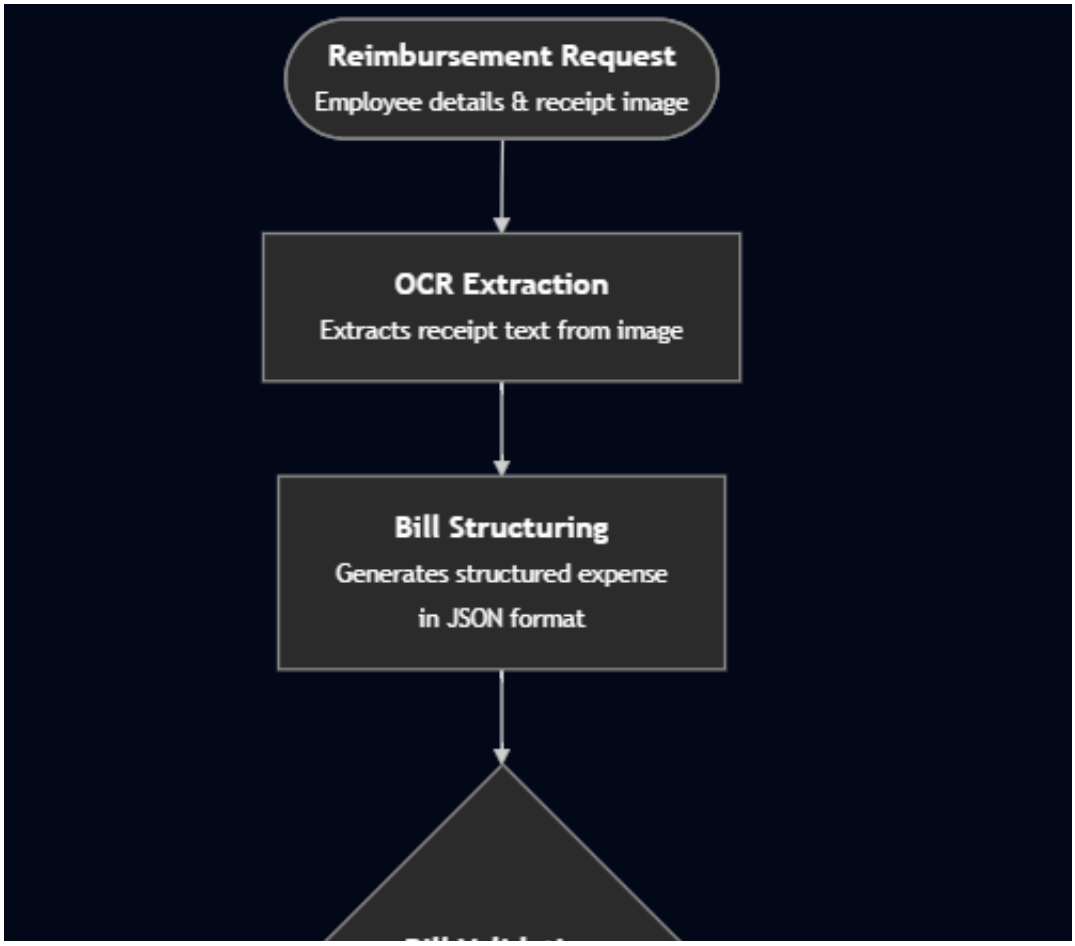
- **Receipt OCR Tool** - Extracts raw text from the uploaded receipt image using a multimodal LLM, preserving the original bill contents as the single source of truth for all downstream processing.
- **Bill Structuring Tool** - Converts the unstructured OCR output into a standardized JSON representation containing transaction metadata and category-wise expense amounts required for reimbursement processing.
- **Bill Validation Tool** - Audits the structured bill against the original receipt, verifying arithmetic consistency, tax inclusion, category allocation, and overall data integrity before any financial decision is made.
- **Bill Re-Structuring Tool** - Activated only when validation fails; repairs the structured bill using the original receipt and the validation rationale, then returns the corrected output for re-validation until a valid expense structure is obtained.
- **Policy Retrieval Tools** - Retrieve reimbursement limits and the employee's historical reimbursement utilization from the enterprise database through MCP-enabled SQL tools, ensuring all reimbursement decisions are based on current organizational policies.
- **Reimbursement Calculation Tool** - Deterministically computes the approved reimbursement for each expense category by applying policy limits and previous utilization, ensuring approved amounts never exceed the

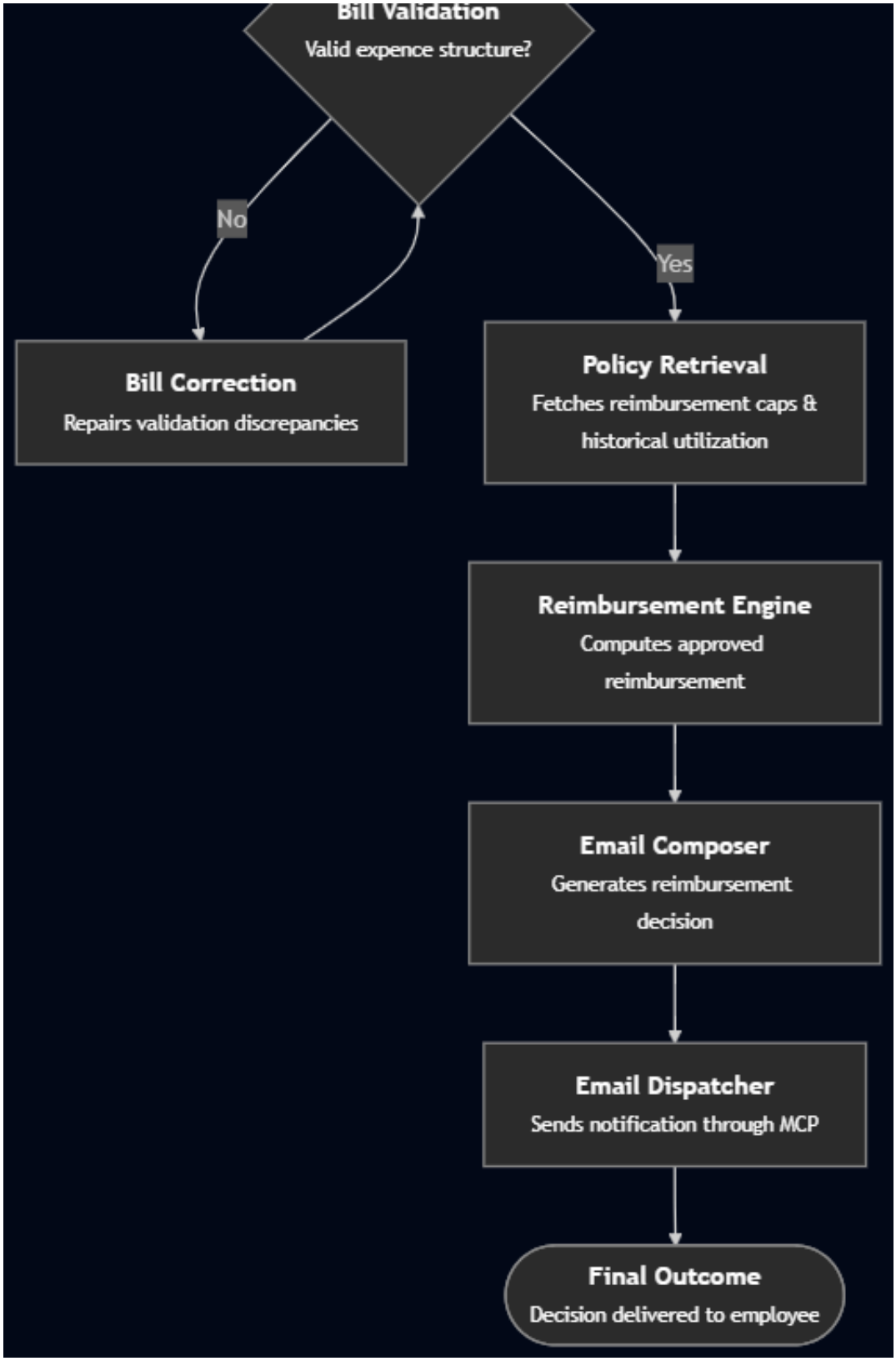
employee's remaining eligibility.

- **Decision Email Generation Tool** - Produces a professional, audit-ready reimbursement decision email summarizing the approved reimbursement amounts together with the policy-based rationale behind the decision.
- **Email Dispatch Tool** - Sends the finalized reimbursement decision through an MCP-managed email service, allowing the agent to perform real-world actions without requiring direct access to external systems.

This architecture closely mirrors how an experienced finance reviewer processes reimbursement requests: extract receipt information, organize the expense details, verify correctness, correct inconsistencies when necessary, apply reimbursement policies, calculate the approved amounts, communicate the decision, and finally notify the employee. By separating reasoning from external actions through MCP tools and combining iterative validation with deterministic reimbursement calculations, the workflow produces transparent, auditable, and policy-compliant reimbursement decisions while remaining easy to extend with additional enterprise business tools in the future.

Flow Diagram





Note:

- This notebook includes prompts for selected code cells to generate the required implementation. Since LLM responses can vary, the generated code and outputs may differ slightly from the reference solution and may

occasionally require minor modifications before execution.

Installing and Importing Necessary Libraries and Dependencies

In [1]:
```
!pip install -qU \
      openai==2.41.1 \
      langchain==1.3.9 \
      langchain-openai==1.3.3 \
      langchain-community==0.4.2 \
      langgraph==1.2.5 \
      fastmcp==2.12.4 \
      mcp==1.23.0 \
      nest_asyncio==1.6.0 \
      2>/dev/null
```

Note:

- After running the above cell, kindly restart the runtime (for Google Colab) or notebook kernel (for Jupyter Notebook), and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in ***this notebook***.

In [ ]:
```
import os
import json
import re
import time
import pandas as pd
import threading
import sqlite3
import asyncio
from typing import Annotated
import base64
import mimetypes
from google.colab import files

import nest_asyncio
nest_asyncio.apply()

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage

from langchain_community.utilities import SQLDatabase
```

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

# Corrected import for LangGraph's prebuilt agent
from langchain.agents import create_agent

from fastmcp import FastMCP
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client as stream

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning, module=
```

```
/tmp/ipykernel_1628/2067030113.py:21: DeprecationWarning: `langchain-co
mmunity` is being sunset and is no longer actively maintained. See http
s://github.com/langchain-ai/langchain-community/issues/674 for details
and migration guidance toward standalone integration packages.
  from langchain_community.utilities import SQLDatabase
/usr/local/lib/python3.12/dist-packages/fastmcp/server/auth/providers/j
wt.py:10: AuthlibDeprecationWarning: authlib.jose module is deprecated,
please use joserfc instead.
It will be compatible before version 2.0.0.
  from authlib.jose import JsonWebKey, JsonWebToken
```

Setting Up Email Credentials and SMTP Configuration

As part of the reimbursement workflow, the system must notify employees about the outcome of their reimbursement requests, which requires controlled access to an email service for sending notifications.

To enable automated reimbursement notifications, the system needs permission to send emails on behalf of a sender account. This can be achieved using **SMTP credentials**, which allow the application to authenticate with an email service and deliver messages securely.

Instead of hardcoding credentials inside the logic, we externalize all email-related settings into a configuration file. This makes the notebook:

- Safer (credentials are not scattered in code)
- Easier to maintain
- Portable across email providers (Gmail, Outlook, Zoho, corporate SMTP)

We use an **App Password** instead of a login password to ensure restricted, audit-

safe access.

Generating an App Password (Gmail)

Most email providers do not allow direct login passwords to be used for automated access. Instead, they require an **App Password**, which is a restricted credential specifically designed for programmatic email access.

For Gmail accounts:

1. Enable **2-Step Verification** in your Google account.
2. Navigate to **App Passwords** under Security settings.
3. Create a new app password for *Mail*.
4. Use the generated password for SMTP authentication.

This approach limits access to email functionality only and improves overall security.

NOTE: While the steps above are shown for Gmail, most other email providers (such as Outlook, Zoho, and Rediffmail) follow a similar approach and require app-specific or service-specific passwords for secure SMTP access.

Storing Email Credentials

Only sensitive credentials required for authentication are stored in the configuration file. These include:

- The OpenAI API key and base URL
- The sender email address
- The app password associated with that email account

All other email behavior settings are configured separately to keep the system flexible and easy to update.

```
{
  "OPENAI_API_KEY": "your-openai-key",
  "OPENAI_API_BASE": "your-openai-base",

  "SENDER_EMAIL": "your-email-id",
  "GMAIL_APP_PASSWORD": "your-app-password"
}
```

SMTP Host and Port Configuration

SMTP host and port define the **email service responsible for delivering messages** from the system.
These settings must match the provider of the sender email account.

If the sender email belongs to a different service, the SMTP host and port must be updated accordingly to ensure successful authentication and email delivery.

Below are commonly used SMTP configurations. Select the configuration that matches the sender email service.

Email Provider	SMTP Host	Port	Encryption
Gmail	smtp.gmail.com	465	SSL
Gmail (STARTTLS)	smtp.gmail.com	587	STARTTLS
Outlook / Office365	smtp.office365.com	587	STARTTLS
Yahoo Mail	smtp.mail.yahoo.com	465	SSL
Zoho Mail	smtp.zoho.com	465	SSL
Corporate Email	Provided by IT Team	Varies	Varies

In this case study, we use a **Gmail sender account authenticated via an App Password**.
Accordingly, the SMTP host and port are configured for **Gmail's SMTP service**.

In [ ]:
```
port=465

SMTP_host='smtp.gmail.com'
```

Email delivery relies on two things:

1. Valid authentication credentials (sender email and app password)
2. Correct SMTP host and port for the email provider

As long as both are correctly configured, the system can send audit-safe, automated notifications without any changes to the core workflow logic.

Model Initialization

OpenAI API Calling

In [ ]:
```
# Load the JSON file and extract values
file_name = 'config.json'
```

```
with open(file_name, 'r') as file:
    config = json.load(file)
    OPENAI_API_KEY = config.get("OPENAI_API_KEY")
    OPENAI_API_BASE = config.get("OPENAI_API_BASE")
    GMAIL_APP_PASSWORD = config.get("GMAIL_APP_PASSWORD")
    SENDER_EMAIL = config.get("SENDER_EMAIL")

# Store API credentials in environment variables
os.environ['OPENAI_API_KEY'] = OPENAI_API_KEY
os.environ["OPENAI_BASE_URL"] = OPENAI_API_BASE
```

In [ ]:
```
llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

MCP Setup

Next, we initialize the MCP instance, which will act as the central control layer for this notebook exposing business tools (like SQL queries and email actions) and enabling structured, tool-based interactions over a controlled interface.

In [ ]:
```
mcp = FastMCP("demo", host="127.0.0.1", port=8000)
```

```
/usr/local/lib/python3.12/dist-packages/fastmcp/server/server.py:255: D
eprecationWarning: Providing `host` when creating a server is deprecate
d. Provide it when calling `run` or as a global setting instead.
  self._handle_deprecated_settings(
/usr/local/lib/python3.12/dist-packages/fastmcp/server/server.py:255: D
eprecationWarning: Providing `port` when creating a server is deprecate
d. Provide it when calling `run` or as a global setting instead.
  self._handle_deprecated_settings(
```

SQL MCP Tool

This tool allows MCP to execute SQL queries on the database in a controlled and structured way.

The `@mcp.tool` decorator registers this function as a callable MCP tool, making it safely accessible to the agent through the MCP server instead of allowing direct database access.

In [ ]:
```
DB_PATH = "/content/reimbursement.db"
```

Prompt:

Create an MCP tool function decorated with @mcp.tool() that executes SQL queries on an SQLite database and returns structured JSON responses. Accept db_path and sql as input parameters.

Connect to the database and handle SELECT/WITH read queries by returning JSON with 'columns' and 'rows', handle write operations by committing and returning a status of 'ok', and catch exceptions to return a JSON error message.

In [ ]:
```python
@mcp.tool()
def run_sql(db_path: str, sql: str) -> str:
    """
    Execute SQL on sqlite db and return JSON:
      {"columns":[...], "rows":[...]}
    or {"error":"..."}.
    """
    try:
        # Establish a connection to the SQLite database
        conn = sqlite3.connect(db_path)
        cur = conn.cursor()

        # Execute the provided SQL query
        cur.execute(sql)

        # Handle read queries (SELECT / WITH)
        if sql.strip().lower().startswith(("select", "with")):
            rows = cur.fetchall()
            cols = [d[0] for d in cur.description] if cur.description
            conn.close()
            return json.dumps({"columns": cols, "rows": rows})
        else:
            # Handle write operations (INSERT / UPDATE / DELETE)
            conn.commit()
            conn.close()
            return json.dumps({"status": "ok"})

    except Exception as e:
        # Return any execution error in a structured format
        return json.dumps({"error": str(e)})
```

EMAIL MCP Tool

Prompt:

Create an MCP tool function decorated with @mcp.tool() that sends an email using Gmail SMTP over SSL and returns a structured JSON response. Accept sender_email, app_password, receiver_email, subject, and body as input parameters. Construct a plain text MIMEMultipart message, connect securely to the SMTP server, authenticate using the provided credentials, and send the message.

Return a JSON string with 'status': 'sent' upon successful delivery, or catch exceptions to return 'error': ''.

This tool enables MCP to send emails programmatically, allowing the system to trigger real-world notifications and actions as part of an automated workflow.

In [ ]:
```
@mcp.tool()
def send_email(
    sender_email: str,
    app_password: str,
    receiver_email: str,
    subject: str,
    body: str
) -> str:
    """
    Sends an email using Gmail SMTP.

    This MCP tool is responsible for delivering the final reimbursemen
    decision email. It authenticates using the sender's email address
    and a Gmail App Password, then sends the message securely over SSL

    Returns:
    - {"status": "sent"} on successful delivery
    - {"error": "<message>"} if email sending fails
    """
    # Log the recipient for traceability during execution
    print(f"[INSIDE TOOL]: send_email to={receiver_email}")

    try:
        # Create the email container
        msg = MIMEMultipart()
        msg["From"] = sender_email          # Sender email address
        msg["To"] = receiver_email          # Recipient email address
        msg["Subject"] = subject             # Email subject line

        # Attach the email body as plain text
        msg.attach(MIMEText(body, "plain"))

        # Establish a secure SSL connection to the SMTP server
        # SMTP_host and port are configured earlier for Gmail
        with smtplib.SMTP_SSL(SMTP_host, port) as server:

            # Authenticate using the app password (not the login passw
            server.login(sender_email, app_password)

            # Send the composed email message
            server.send_message(msg)

        # Return a structured success response
        return json.dumps({"status": "sent"})
```

```
    except Exception as e:
        # Catch and return any SMTP or authentication errors
        return json.dumps({"error": str(e)})
```

Starting the MCP Server: Making Tools Callable

Before our system can run SQL queries or send emails, we need a central service that exposes these capabilities in a structured and controlled way. This is where the MCP (Model Context Protocol) server comes in.

Think of the MCP server as:

- A tool hub that hosts business capabilities (SQL, email, etc.)
- A bridge between our notebook (or an AI agent) and real-world actions
- A controlled execution layer, instead of running arbitrary code directly

To keep the notebook interactive, we start the MCP server in the background.

Starting the MCP Server in a Background Thread

Here, we configure MCP to run over streamable HTTP, which is required for:

- Long-lived connections
- Tool calls with structured responses
- Two-way communication between client and server

In [ ]:
```
def start_mcp_server():
    """
    Starts the MCP server using streamable HTTP transport.

    streamable-http allows the MCP server to:
    - Accept HTTP-based streaming connections
    - Support bidirectional communication (read/write streams)
    """
    mcp.run(transport="streamable-http")
```

We run the server in a daemon thread so:

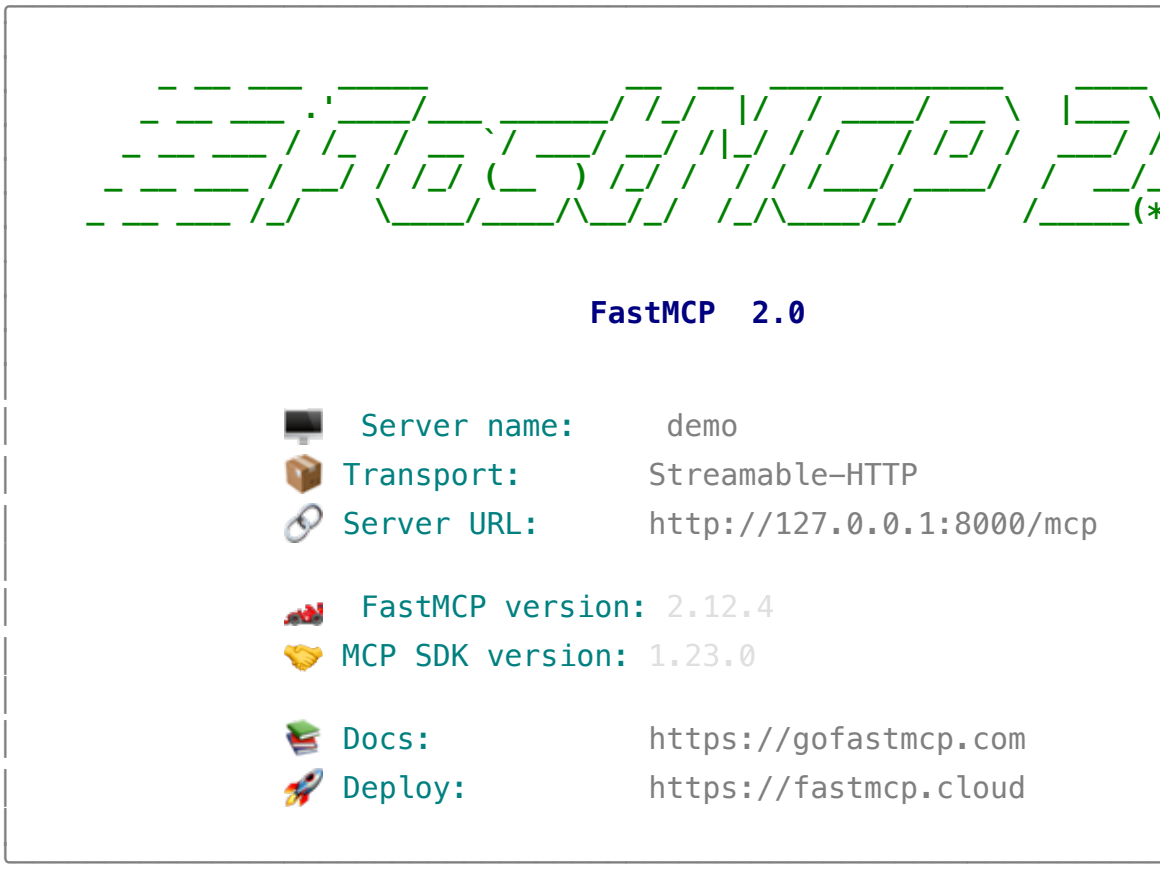
- The notebook does not freeze
- The server shuts down automatically when the notebook stops
- We can continue executing client-side code in later cells

In [ ]:
```python
# Start the MCP server in a background thread so that
# it does not block the Jupyter notebook execution
server_thread = threading.Thread(
    target=start_mcp_server,
    daemon=True  # Daemon thread exits automatically with the main pro
)

server_thread.start()

# Give the server a moment to initialize
time.sleep(2)

print("MCP server started on http://127.0.0.1:8000/mcp")
```




```
[07/02/26 09:13:33] INFO     Starting MCP server 'demo' with tran
                             http://127.0.0.1:8000/mcp
```

```
/usr/local/lib/python3.12/dist-packages/websockets/legacy/__init__.py:
6: DeprecationWarning: websockets.legacy is deprecated; see https://web
sockets.readthedocs.io/en/stable/howto/upgrade.html for upgrade instruc
tions
  warnings.warn(  # deprecated in 14.0 - 2024-11-09
/usr/local/lib/python3.12/dist-packages/uvicorn/protocols/websockets/we
bsockets_impl.py:17: DeprecationWarning: websockets.server.WebSocketSer
verProtocol is deprecated
  from websockets.server import WebSocketServerProtocol
INFO:     Started server process [1628]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to qui
t)
MCP server started on http://127.0.0.1:8000/mcp
```

Defining How the Client Connects to MCP

Now that the MCP server is running, the client needs to know where to find it.

In [ ]:
```python
# MCP server endpoint
MCP_URL = "http://127.0.0.1:8000/mcp"
```

This endpoint acts as the single entry point for all tool-based interactions that follow.

Helper Function: Picking Read & Write Streams

When we connect to the MCP server using a streaming HTTP client, the connection returns multiple internal objects, not all of which are directly useful.

However, MCP communication fundamentally relies on:

- A write stream -> to send requests
- A read stream -> to receive responses

This helper function extracts the correct streams safely.

In [ ]:
```python
def _pick_streams(ctx_tuple):
    """
    streamable_http_client yields a tuple/list with multiple items.
    We pick:
      - write_stream: has .send
      - read_stream: has .receive
    """
    write_stream = None
    read_stream = None
```

```
    for obj in ctx_tuple:
        if write_stream is None and hasattr(obj, "send"):
            write_stream = obj
        if read_stream is None and hasattr(obj, "receive"):
            read_stream = obj

    if read_stream is None or write_stream is None:
        raise RuntimeError(
            "Could not find read/write streams in streamable_http_clie
            f"Got types: {[type(x) for x in ctx_tuple]}"
        )
    return read_stream, write_stream
```

This makes the connection logic explicit and robust, instead of relying on assumptions about object order.

Core MCP Interaction Logic (Async)

Prompt:

Implement a notebook-safe wrapper function that calls an async MCP tool by name with arguments and returns a dictionary response without raising event loop errors.

This function is the **heart of our MCP-based system**. Its job is to take a tool name (for example, `run_sql` or `send_email`) along with input arguments, connect to the running MCP server, and execute that tool in a safe and structured way.

Conceptually, this function performs four key steps:

1. **Connect to the MCP server** using a streaming HTTP client.
2. **Establish a session** by identifying read/write streams and completing a protocol handshake.
3. **Invoke the requested tool** with the provided arguments.
4. **Parse and return the result** in a clean, predictable format (JSON when possible).

Because MCP communication is asynchronous and stream-based by design, this function is written as `async`. A synchronous wrapper is added later so notebook users can call tools without worrying about async details.

In [ ]:
```
async def _mcp_tool_call_async(tool_name: str, args: dict) -> dict:
    # Step 1: Open a streaming HTTP connection to the MCP server
```

```
    async with streamable_http_client(MCP_URL) as ctx:

        # Ensure the client returns multiple stream-related objects
        # (MCP requires both a read stream and a write stream)
        if not isinstance(ctx, (tuple, list)):
            raise RuntimeError(
                f"Expected tuple/list from streamable_http_client, got
            )

        # Step 2: Extract the correct read and write streams needed fo
        read_stream, write_stream = _pick_streams(ctx)

        # Step 3: Create a client session using the extracted streams
        async with ClientSession(read_stream, write_stream) as session

            # Initialize the session to complete the MCP handshake
            # and make registered tools available
            await session.initialize()

            # Step 4: Call the specified MCP tool with the provided ar
            res = await session.call_tool(tool_name, args)

            # Step 5: Attempt to parse structured text output (usually
            if res.content and hasattr(res.content[0], "text"):
                txt = res.content[0].text
                try:
                    return json.loads(txt)
                except Exception:
                    # If parsing fails, return the raw text safely
                    return {"raw_text": txt}

            # Final fallback for unexpected response formats
            return {"raw": str(res)}
```

Notebook-Safe Wrapper (Why This Exists)

Jupyter notebooks already run an event loop. Calling async functions directly can cause errors like:

“RuntimeError: event loop is already running”

To avoid this, we expose a synchronous wrapper.

In [ ]:
```
def mcp_tool_call(tool_name: str, args: dict) -> dict:
    # safe in notebooks because nest_asyncio.apply() is active
    return asyncio.run(_mcp_tool_call_async(tool_name, args))
```

This allows learners to call MCP tools like normal Python functions.

Tool Wrapper: Run SQL via MCP

Finally, we define task-specific helpers that hide protocol details and expose intent clearly.

In [ ]:
```
def mcp_run_sql(sql: str) -> dict:
    return mcp_tool_call("run_sql", {"db_path": DB_PATH, "sql": sql})
```

Tool Wrapper: Send Email via MCP

This enables real-world actions while keeping credentials and execution logic outside the model.

In [ ]:
```
def mcp_send_email(to_email: str, subject: str, body: str) -> dict:
    return mcp_tool_call(
        "send_email",
        {
            "sender_email": SENDER_EMAIL,
            "app_password": GMAIL_APP_PASSWORD,
            "receiver_email": to_email,
            "subject": subject,
            "body": body,
        },
    )
```

Agentic AI Implementation

In [ ]:
```
import socket

def port_in_use(port, host="127.0.0.1"):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        return s.connect_ex((host, port)) == 0

if not port_in_use(8000):
    server_thread = threading.Thread(target=start_mcp_server, daemon=T
    server_thread.start()
    time.sleep(2)
    print("MCP server started on http://127.0.0.1:8000/mcp")
else:
    print("MCP server already running on http://127.0.0.1:8000/mcp")
```

```
MCP server already running on http://127.0.0.1:8000/mcp
```

In [ ]:
```
BILLS_FOLDER = "/content"
```

Tool 1: Extract Bill from Image

Prompt:

Implement a tool that reads a bill or receipt image from the bills folder, uses a multimodal LLM to extract all visible text, and returns the raw OCR-transcribed content exactly as it appears on the bill; preserving line breaks, line items, amounts, and taxes without any reformatting or interpretation.

Before the agent can structure or validate anything, it needs the **raw text** of the bill.

This tool reads the image and asks the multimodal LLM (`gpt-4o`) to transcribe the receipt verbatim. It returns the raw OCR-style text, which the agent then feeds into `structure_bill` in the next step.

In [ ]:
```python
@tool
def extract_bill_from_image(image: str) -> str:
    """
    Read a bill / receipt image from the local bills folder and return
    raw text (OCR-style transcription) using the multimodal LLM.

    Use this tool FIRST, before structuring the bill. It converts the
    receipt image into the raw text that all downstream tools expect.

    Args:
        image: The bill image filename (e.g. "bill_001.png") located i
               `bills/` folder. A full path is also accepted.

    Returns:
        A string containing the raw bill text transcribed from the ima
        (line breaks, item rows, amounts, taxes — all preserved as-is)
    """
    image_value = str(image).strip()
    if os.path.isabs(image_value) or os.path.sep in image_value:
        image_path = image_value
    else:
        image_path = os.path.join(BILLS_FOLDER, image_value)

    mime, _ = mimetypes.guess_type(image_path)
    if mime is None:
        mime = "image/png"

    with open(image_path, "rb") as f:
        b64 = base64.b64encode(f.read()).decode("utf-8")
    data_url = f"data:{mime}; base64,{b64}"

    ocr_prompt = (
        "You are an OCR engine. Extract ALL the text visible in this b
        "receipt image, preserving the original layout, line breaks, i
        "and numbers exactly as they appear. Do not summarize, do not
```

```
        "commentary, do not wrap in code fences. Return only the raw "
        "transcribed text of the bill."
    )

    message = HumanMessage(
        content=[
            {"type": "text", "text": ocr_prompt},
            {"type": "image_url", "image_url": {"url": data_url}},
        ]
    )
    response = llm.invoke([message])
    return response.content
```

Tool 2: Structure a Raw Bill

Prompt:

Implement a tool that takes raw OCR bill text, analyzes it using an LLM to identify the transaction date, vendor information (name, address, invoice number), and categorizes all expenses by type (food and beverages, travel and transport, hotel and lodging, other miscellaneous items), then returns a structured summary that shows each category's amount and the total bill, including all applicable taxes ready for audit and validation checks.

Bill Structuring Tool: This tool converts the raw OCR bill text into a clean, standardized expense breakdown (date, vendor details, category-wise amounts, and total) that can be reliably used for policy checks and reimbursement calculations.

In [ ]:
```
@tool
def structure_bill(raw_bill: str) -> str:
    """
    Convert raw OCR bill text into a structured JSON breakdown.

    Use this tool FIRST when you receive a new reimbursement request.
    It extracts the date, vendor, and category-wise (food/travel/hotel
    amounts from the unstructured bill text.

    Args:
        raw_bill: The raw OCR text of the bill / receipt.

    Returns:
        A JSON string with keys: date, vendor_details, food_bill_amoun
        travel_bill_amount, hotel_bill_amount, others_bill_amount,
        total_bill_amount.
    """
```

```
    structuring_prompt = f"""

### ROLE
You are a senior Corporate expense analyst specializing in vendor-agno
hotels, restaurants, and travel agencies.


### INPUT

Raw bill text extracted from receipts or invoices.

Raw Bill Text:
{raw_bill}


### OBJECTIVE

Extract and compute only the following seven outputs from the raw bill

1. Date

   * Invoice or transaction date
   * If unavailable, return `null`

2. Vendor Details (Single String)

   * A single string combining:
     * Vendor name
     * Vendor address
     * Invoice or receipt number
   * Format the string clearly and consistently, for example:
     `"Vendor Name | Address | Invoice #12345"`
   * If vendor details are not identifiable, return `null`

3. Food Bill Amount

   * Total for food, beverages, and liquor + the tax associated with t
   * If none present, return `0`

4. Travel Bill Amount

   * Total for flights, taxis, rideshare, rental cars + the tax associ
   * If none present, return `0`

5. Hotel Bill Amount

   * Total for room charges, lodging fees, occupancy taxes + the tax a
   * If none present, return `0`

6. Others Total Amount

   * Total for parking, tips, laundry, internet, or ambiguous items +
```

```
   * If none present, return `0`

7. Total Bill amount
   * Final total explicitly stated on the bill that should be equal to

### CATEGORIZATION RULES (UNIVERSAL)

* Hotel: room charges, lodging fees, occupancy taxes
* Food: meals, beverages, catering, room service food
* Travel: flights, taxis, rideshare, rental cars, travel agency charge
* Others: parking, tips, laundry, internet, ambiguous items

If classification is ambiguous, assign to Others.


### OUTPUT FORMAT (STRICT JSON ONLY)

{{
  "date": "YYYY-MM-DD or null",
  "vendor_details": "string or null",
  "food_bill_amount": number,
  "travel_bill_amount": number,
  "hotel_bill_amount": number,
  "others_bill_amount": number,
  "total_bill_amount": number
}}


### CONSTRAINTS

* Output must be **valid JSON only**
* No markdown, explanations, or comments
* Do **not** guess missing values
* Use `null` for missing textual data
* Use `0` for missing monetary values
* All monetary values must be tax-inclusive
* Be conservative and audit-safe

"""

    output = llm.invoke(structuring_prompt).content.strip()
    # Extract the JSON block and re-serialize to ensure clean JSON for
    json_text = re.search(r"\{.*\}", output, re.DOTALL).group()
    return json_text
```

Tool 3: Validate a Structured Bill (Guardrail)

Prompt:

Implement a guardrail tool that acts as an expense auditor,

comparing the structured bill summary against the raw OCR text to verify that every claimed amount can be traced back to explicit line items and taxes on the original bill. Check that the categorization makes sense, that totals add up correctly, and that no data was invented or inferred. Return a validation decision (approved or blocked) with a clear explanation of what passed or failed; if blocked, identify the specific numeric or category mismatch.

Structuring Guardrail Tool: This tool acts as a strict audit checkpoint, validating that the structured expense amounts are fully evidence-backed, tax-inclusive, and arithmetically consistent with the original bill before allowing further reimbursement processing.

In [ ]:
```
@tool
def validate_structured_bill(raw_bill: str, structured_bill_json: str)
    """
    Audit a structured bill against the raw bill for arithmetic and
    tax-inclusion consistency.

    Use this tool AFTER structure_bill (or restructure_bill) to confir
    the structuring is correct before proceeding to policy checks. If
    the decision is BLOCK, you must call restructure_bill with the
    returned rationale, then validate again.

    Args:
        raw_bill: The original raw OCR bill text (source of truth).
        structured_bill_json: The structured JSON produced by structur

    Returns:
        A JSON string: {"decision": "APPROVE" or "BLOCK", "rationale":
    """
    guardrail_prompt = f"""
ROLE:
You are a senior expense compliance auditor. Your role is to check the
for each category from the raw bill text.

INPUTS:
1. Raw bill text (source of truth):
{raw_bill}

2. Structured expense data to validate:
{structured_bill_json}

VALIDATION OBJECTIVE:
Approve expense data if all category amounts can be
EXACTLY derived from explicit bill line items and taxes
present in the raw bill.
```

```
DETERMINISTIC ACCEPTANCE RULES (ALL MUST PASS):

1. Evidence-based recomposition (CRITICAL):
   - Category totals MAY be computed by summing:
     (base charges + explicitly labeled taxes from the raw bill)
   - Do this reasoning category by category, then produce the answer
   - Arithmetic recomposition is allowed and expected

2. Tax inclusion:
   - All category totals MUST include applicable taxes
   - Taxes are validated only from the raw bill text
   - Structured data is validated only at the final category total lev
   - If tax attribution from the raw bill is unclear → BLOCK

3. Category integrity:
   - Food = food + beverage + liquor + liquor tax
   - Hotel = room charges + lodging fees + occupancy taxes
   - Travel = transport charges + transport taxes
   - Other = laundry, parking, and taxes not covered above

4. Arithmetic consistency (STRICT):
   - Each structured category total MUST exactly equal its recomposed
   - food + travel + hotel + other MUST equal total_bill_amount
   - Exact match required (no tolerance or rounding assumptions)

DECISION RULE:
- APPROVE if ALL rules pass
- BLOCK if ANY rule fails

RATIONALE RULES:
- State the precise validation reason
- If BLOCKED, identify the exact category or numeric mismatch
- If APPROVED, state that totals are deterministically derived and tax

OUTPUT FORMAT (STRICT JSON ONLY):
{{
  "decision": "APPROVE" or "BLOCK",
  "rationale": "string"
}}

CONSTRAINTS:
- JSON only
- No corrections or suggestions
- Conservative, arithmetic-first validation

"""
    output = llm.invoke(guardrail_prompt).content.strip()
    json_text = re.search(r"\{.*\}", output, re.DOTALL).group()
    return json_text
```

Tool 4: Re-Structure a Blocked Bill

Prompt:

Implement a tool that takes a previously rejected bill structure and the auditor's feedback about what went wrong, then re-analyzes the raw OCR text to fix only the identified issues. Do not invent missing data or make changes unrelated to the auditor's feedback. Return a corrected version of the bill summary in the same structured format, ready for another validation pass.

Bill Re-structuring Tool: This tool repairs only the specific errors flagged by the audit guardrail, using the raw bill as the single source of truth to produce a corrected, tax-inclusive, and arithmetically consistent expense breakdown.

In [ ]:
```
@tool
def restructure_bill(raw_bill: str, previous_structured_json: str, blo
    """
    Repair a previously BLOCKED structured bill, using the auditor's
    rationale to correct only the flagged errors.

    Use this tool when validate_structured_bill returns decision = BLO
    After calling this, you must call validate_structured_bill again t
    confirm the corrections passed.

    Args:
        raw_bill: The original raw OCR bill text.
        previous_structured_json: The blocked structured JSON.
        block_rationale: The auditor's rationale explaining why it was

    Returns:
        A corrected structured bill JSON (same schema as structure_bil
    """
    restructuring_prompt = f"""
ROLE:
You are a senior Corporate Expense Analyst specializing in vendor-agno
across hotels, restaurants, and travel agencies.

You are correcting a previously BLOCKED expense structure.
Your responsibility is to FIX only the identified errors using bill ev
You must NOT invent, infer, or enhance data.

INPUTS:
1. Raw bill text (single source of truth):
{raw_bill}

2. Previously structured expense data (BLOCKED):
{previous_structured_json}

3. Auditor rationale explaining why the data was BLOCKED:
```

```
{block_rationale}

OBJECTIVE:
Re-structure the expense data by correcting ONLY the issues explicitly
in the auditor rationale, while keeping all other correct fields uncha

You must produce a corrected version of the same structured output sch

CORRECTION PRINCIPLES (STRICT):

1. Source of truth:
   - The raw bill text is the ONLY authoritative source.
   - Do not rely on assumptions, common patterns, or inferred totals.

2. Scope of correction:
   - Modify ONLY the fields flagged as incorrect in the auditor ration
   - Do NOT alter fields that are already correct.

3. Tax handling (MANDATORY):
   - ALL monetary values must be tax-inclusive.
   - If tax is listed separately, add it to the appropriate category:
     - Food → food-related taxes (e.g., liquor tax, service tax on mea
     - Hotel → occupancy tax, lodging tax
     - Travel → transport-related taxes
   - If tax attribution is unclear, include it under Others.

4. Categorization rules (UNIVERSAL):
   - Hotel: room charges, lodging fees, occupancy taxes
   - Food: meals, beverages, liquor, catering, room service food
   - Travel: flights, taxis, rideshare, rental cars, travel agency cha
   - Others: parking, tips, laundry, internet, service fees, ambiguous
   - If classification is ambiguous, assign to Others.

5. Verification and safety:
   - Do not introduce new vendors, dates, or invoice numbers.
   - Do not change amounts unless they are explicitly incorrect per th
   - If a value cannot be confidently verified from the bill, use:
     - null for text fields
     - 0 for monetary fields

6. Arithmetic consistency:
   - Category totals must not exceed bill evidence.
   - food + travel + hotel + others MUST equal total_bill_amount.
   - total_bill_amount must exactly match the final total stated on th
   - If the total is missing from the bill, return 0.

OUTPUT FORMAT (STRICT  JSON ONLY):
{{
  "date": "YYYY-MM-DD or null",
  "vendor_details": "string or null",
  "food_bill_amount": number,
  "travel_bill_amount": number,
```

```
  "hotel_bill_amount": number,
  "others_bill_amount": number,
  "total_bill_amount": number
}}

CONSTRAINTS:
- Output valid JSON only
- No markdown, explanations, or comments
- Do not guess missing values
- Conservative, audit-safe corrections only
"""
    output = llm.invoke(restructuring_prompt).content.strip()
    json_text = re.search(r"\{.*\}", output, re.DOTALL).group()
    return json_text
```

Tool 5: Get Reimbursement Caps (Policy Lookup)

Prompt:

Implement a tool that looks up the company reimbursement policy for an employee based on their position, retrieves the maximum reimbursable amounts for each expense category (food and meals, travel and transport, hotel and lodging), and returns a structured summary of these policy limits; defaulting to zero if no policy is found for that employee.

Reimbursement Cap Retrieval Tool: This tool determines the maximum reimbursable limits for food, travel, and hotel expenses for a specific employee by querying database.

In [ ]:
```
@tool
def get_reimbursement_caps(emp_id: str) -> str:
    """
    Fetch the maximum reimbursable amount per category (food, travel,
    for an employee, based on their position and company reimbursement

    Use this BEFORE calculating reimbursement amounts.

    Args:
        emp_id: The employee identifier.

    Returns:
        A JSON string: {"food": <cap>, "travel": <cap>, "hotel": <cap>
    """
    sql_query = f""" SELECT category, reimbursement_cap AS cap_amount
                     (SELECT position FROM employee_details WHERE emp
    mcp_res = mcp_run_sql(sql_query)
```

```
    caps = {"food": 0.0, "travel": 0.0, "hotel": 0.0}
    if mcp_res and "rows" in mcp_res:
        for row in mcp_res["rows"]:
            if len(row) >= 2:
                cat = str(row[0]).strip().lower()
                if cat in caps:
                    caps[cat] = float(row[1] or 0)
    return json.dumps(caps)
```

Tool 6: Get Past Utilization

Prompt:

Implement a tool that queries the reimbursement history to calculate how much money the employee has already been reimbursed in each expense category across all their past approved submissions, then returns a summary of this cumulative usage per category; this will be compared against policy limits to determine remaining budget.

Previous Utilization Lookup Tool: This tool fetches how much reimbursement the employee has already used in each category, providing the necessary context to prevent over-allocation beyond allowed limits.

In [ ]:
```
@tool
def get_past_utilization(emp_id: str) -> str:
    """
    Fetch how much reimbursement the employee has already used per cat
    summed across their full reimbursement history.

    Use this AFTER get_reimbursement_caps so you have both the caps an
    the past usage before calculating final approved amounts.

    Args:
        emp_id: The employee identifier.

    Returns:
        A JSON string: {"food": <used>, "travel": <used>, "hotel": <us
    """
    sql_query = f"""SELECT category, SUM(amount) AS total_amount FROM
    mcp_res = mcp_run_sql(sql_query)

    used = {"food": 0.0, "travel": 0.0, "hotel": 0.0}
    if mcp_res and "rows" in mcp_res:
        for row in mcp_res["rows"]:
            if len(row) >= 2:
```

```
                cat = str(row[0]).strip().lower()
                if cat in used:
                    used[cat] = float(row[1] or 0)
    return json.dumps(used)
```

Tool 7: Calculate Approved Reimbursement

Prompt:

Implement a tool that takes the requested bill amounts per category, the policy caps, and the employee's past usage, then calculates for each expense type how much can actually be reimbursed (requested amount but not exceeding the available remaining budget). Return the calculation results showing the requested amount, remaining policy budget, and approved amount for each category, ensuring the logic is deterministic and audit-safe.

Reimbursement Calculation Tool: This tool applies policy caps and past utilization to compute the final approved reimbursement amounts for each category, ensuring approvals never exceed remaining eligible limits.

In [ ]:
```
@tool
def calculate_approved_reimbursement(
    food_bill_amount: float,
    travel_bill_amount: float,
    hotel_bill_amount: float,
    food_cap: float,
    travel_cap: float,
    hotel_cap: float,
    food_used: float,
    travel_used: float,
    hotel_used: float,
) -> str:
    """
    Deterministically compute the approved reimbursement per category,
    given the requested amounts, the policy caps, and the past utiliza

    For each category: approved = min(requested, max(0, cap - used)).

    Use this AFTER get_reimbursement_caps and get_past_utilization, an
    AFTER the structured bill has been APPROVED by validate_structured

    Returns:
        A JSON string with per-category breakdown:
        {
```

```
          "food":   {"requested_amount": ..., "remaining_cap": ..., "a
          "travel": {"requested_amount": ..., "remaining_cap": ..., "a
          "hotel":  {"requested_amount": ..., "remaining_cap": ..., "a
      }
  """
  food_remaining = max(0.0, food_cap - food_used)
  travel_remaining = max(0.0, travel_cap - travel_used)
  hotel_remaining = max(0.0, hotel_cap - hotel_used)

  summary = {
      "food": {
          "requested_amount": food_bill_amount,
          "remaining_cap": food_remaining,
          "approved_amount": min(food_bill_amount, food_remaining),
      },
      "travel": {
          "requested_amount": travel_bill_amount,
          "remaining_cap": travel_remaining,
          "approved_amount": min(travel_bill_amount, travel_remainin
      },
      "hotel": {
          "requested_amount": hotel_bill_amount,
          "remaining_cap": hotel_remaining,
          "approved_amount": min(hotel_bill_amount, hotel_remaining)
      },
  }
  return json.dumps(summary)
```

Tool 8: Generate Decision Email

Prompt:

Implement a tool that composes a professional reimbursement decision email to the employee. Take the calculation results and employee details as input, draft an audit-safe email subject line and body that clearly explains which expense categories were approved and which were denied (and why), with exact amounts for transparency. The email should be ready to send and include all necessary context for the employee to understand the decision.

Decision Email Generation Tool: This tool converts reimbursement calculation results into an audit-safe reimbursement decision email, generating a professional subject and body that clearly communicate approved amounts and policy-based rationale.

In [ ]:
```
@tool
def generate_decision_email(
```

```
    emp_id: str,
    emp_name: str,
    transaction_date: str,
    vendor_details: str,
    calculation_summary_json: str,
    others_bill_amount: float,
    total_bill_amount: float,
) -> str:
    """
    Compose a professional, audit-safe reimbursement decision email.
    Internally:
      1. Drafts an audit-safe rationale from the calculation summary.
      2. Generates the email subject and body around that rationale.

    Use this AFTER calculate_approved_reimbursement.

    Returns:
        A JSON string: {"subject": "...", "mail_text": "..."}.
    """
    summary = json.loads(calculation_summary_json)
    p = summary

    # Step A: rationale
    rationale_prompt = f"""
ROLE
You explain reimbursement decisions. Do NOT change numbers.

Food:   Requested = {p["food"]["requested_amount"]}, Remaining Cap = {
Travel: Requested = {p["travel"]["requested_amount"]}, Remaining Cap =
Hotel:  Requested = {p["hotel"]["requested_amount"]}, Remaining Cap =

Write an audit-safe rationale ONLY for categories where requested_amou
"""
    rationale = llm.invoke(rationale_prompt).content.strip()

    # Step B: email
    email_prompt = f"""
ROLE: You are a corporate Finance communication specialist.

TASK: Generate a professional and audit-safe reimbursement decision em

INPUT DATA:
- Employee ID: {emp_id}
- Employee Name: {emp_name}
- Transaction Date: {transaction_date}
- Vendor Details: {vendor_details}
- Approved Rationale: {rationale}
- Non-reimbursable Amount (Others): {others_bill_amount}
- Total Bill Amount Submitted: {total_bill_amount}


EMAIL REQUIREMENTS:
```

```
1. Greet the employee by name
2. Reference transaction date, and vendor details
3. Clearly state the reimbursement decision
4. Include the provided rationale in a neutral, audit-safe manner
5. End with a professional Finance closing

TONE AND CONSTRAINTS:
- Professional, neutral, and factual
- No casual language, no apologies
- No assumptions or recalculations
- Suitable for audit and compliance review

OUTPUT FORMAT (STRICT JSON ONLY):
{{
  "subject": "string",
  "mail_text": "string"
}}

CONSTRAINTS:
- Output valid JSON only
- Do not include markdown, explanations, or extra text

"""
    output = llm.invoke(email_prompt).content.strip()
    json_text = re.search(r"\{.*\}", output, re.DOTALL).group()
    return json_text
```

Tool 9: Send the Decision Email

Prompt:

Implement a tool that sends the reimbursement decision email to the employee's email address via the organization's email service, then returns a confirmation status indicating whether the email was successfully delivered, including relevant metadata like the send timestamp for record-keeping.

Email Dispatch Tool: This Tool sends the finalized reimbursement decision email through MCP, completing the workflow by triggering a real-world notification in a controlled and auditable manner.

In [ ]:
```
@tool
def send_decision_email(receiver_email: str, subject: str, mail_text:
    """
    Send the reimbursement decision email to the employee via SMTP (th

    Use this as the FINAL step, after generate_decision_email.
```

```
    Args:
        receiver_email: The employee's email address.
        subject: The email subject line.
        mail_text: The full email body.

    Returns:
        A JSON status string from the MCP email tool.
    """
    status = mcp_send_email(receiver_email, subject, mail_text)
    return json.dumps(status)
```

Assembling the Agent

Tool Registry

We gather all the tools into a list. This list is the agent's full vocabulary of actions - LangGraph binds each tool's name, docstring, and argument schema directly to the LLM, so the agent always has access to this information at every step without it needing to be repeated in the system prompt.

In [ ]:
```
tools = [
    extract_bill_from_image,
    structure_bill,
    validate_structured_bill,
    restructure_bill,
    get_reimbursement_caps,
    get_past_utilization,
    calculate_approved_reimbursement,
    generate_decision_email,
    send_decision_email,
]

for t in tools:
    print(f"- {t.name}: {t.description.splitlines()[0]}")
```

```
- extract_bill_from_image: Read a bill / receipt image from the local b
ills folder and return its
- structure_bill: Convert raw OCR bill text into a structured JSON brea
kdown.
- validate_structured_bill: Audit a structured bill against the raw bil
l for arithmetic and
- restructure_bill: Repair a previously BLOCKED structured bill, using
the auditor's
- get_reimbursement_caps: Fetch the maximum reimbursable amount per cat
egory (food, travel, hotel)
- get_past_utilization: Fetch how much reimbursement the employee has a
lready used per category,
- calculate_approved_reimbursement: Deterministically compute the appro
ved reimbursement per category,
- generate_decision_email: Compose a professional, audit-safe reimburse
ment decision email.
- send_decision_email: Send the reimbursement decision email to the emp
loyee via SMTP (through MCP).
```

The System Prompt

This prompt is the **heart of the agent**. The agent is expected to:

- Read the SOP
- Decide which tool to call next at each step (Thought → Action)
- React to tool outputs (Observation)
- Loop back to re-structure on a BLOCK
- Stop when the email has been sent

Notice the SOP is written declaratively, not imperatively. We tell the agent **what good looks like**, not **what to do at every branch**. The agent's reasoning fills in the rest.

In [ ]:
```
AGENT_SYSTEM_PROMPT = """
You are a Reimbursement Automation Agent. Your job is to process one
employee reimbursement request end-to-end using the tools available to

You will be given: emp_id, emp_name, image, receiver_email

Each tool's description tells you what it does, when to use it, and wh
it expects as input/output - use those descriptions to decide which to
to call next and in what order, based on the outcome of each previous

GOAL:
Take the submitted bill image through extraction, structuring, validat
policy-based reimbursement calculation, and finally notify the employe
email with the decision.

RULES:
- Never invent numbers; only use tool outputs.
```

```
- If a structured bill is BLOCKED during validation, correct it and
  re-validate. Retry at most TWICE; if still BLOCKED after two attempt
  stop and report the rationale instead of proceeding further.
- Reimbursement amounts must only be calculated after both the policy
  and the employee's past utilization have been retrieved.
- The bill must be validated (APPROVE) before any reimbursement is cal
- Sending the decision email is always the final action.

After completing the request (or stopping on an unresolved BLOCK), sum
decision per category, total approved amount, and email status.
"""
```

Create the Agent

`create_agent` is configured with three arguments:

- **model** - the LLM (`llm`) that powers the agent's reasoning and decision-making
- **tools** - the list of registered tools (`tools`) the agent is allowed to call
- **system_prompt** - the operating instructions (`AGENT_SYSTEM_PROMPT`) that define the agent's role, the step-by-step SOP, and the rules it must follow

In [ ]:
```
reimbursement_agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt=AGENT_SYSTEM_PROMPT,
)
```

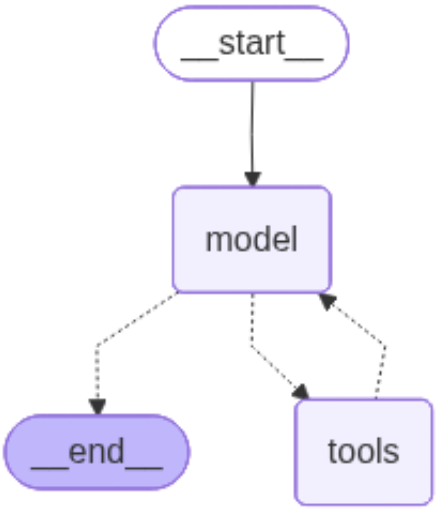
- **reimbursement_agent** - an agent that combines an LLM model with reimbursement tools. The LLM decides what to do, and the tools perform the required actions.

The graph loops between these two steps until the LLM decides no further tool calls are needed.

Visualize the Agent Graph

In [ ]:
```
from IPython.display import Image, display

try:
    display(Image(reimbursement_agent.get_graph().draw_mermaid_png()))
except Exception:
    pass
```



Helper: Run the Agent on One Request

Prompt:

Create a function that runs an end-to-end reimbursement processing workflow using an AI agent. Accept an optional verbose boolean parameter defaulting to True. Interactively prompt the user for employee ID and name, upload a bill image using Google Colab file upload, and construct a task prompt containing employee details and the bill file path. Invoke reimbursement_agent with a HumanMessage, print a formatted execution trace showing tool actions, observations, and reasoning if verbose is enabled, and return the agent's execution result.

We wrap `reimbursement_agent.invoke` in a small helper that:

1. Formats the user task as a single message
2. Invokes the agent
3. Prints the full trace (Thought / Action / Observation) so learners can see what the agent decided at each step

This trace visibility is the whole pedagogical point of using a single agent over a hardcoded state machine.

In [ ]:
```
def run_reimbursement_agent(verbose=True):
    """
    Run the agent on one reimbursement request and optionally
    print the full reasoning + tool-call trace.
    """
```

```
    emp_id = input("Enter employee ID: ")
    emp_name = input("Enter employee name: ")
    print("Upload the Bill")
    image = list(files.upload().keys())[0]
    full_image_path = os.path.join(BILLS_FOLDER, image) # Construct th

    user_task = f"""
Process this reimbursement request end-to-end:

Emp_ID: {emp_id}
emp_name: {emp_name}
receiver_email: {receiver_email}

image:
{full_image_path}
"""

    result = reimbursement_agent.invoke({
        "messages": [HumanMessage(content=user_task)]
    })

    if verbose:
        print("=" * 70)
        print("AGENT TRACE")
        print("=" * 70)
        for m in result["messages"]:
            # AIMessage with tool_calls -> agent decided to call a too
            if hasattr(m, "tool_calls") and m.tool_calls:
                for tc in m.tool_calls:
                    print(f"\n[ACTION] {tc['name']}")
                    args_preview = json.dumps(tc.get("args", {}), inde
                    if len(args_preview) > 400:
                        args_preview = args_preview[:400] + "... <trun
                    print(f"[ACTION INPUT]\n{args_preview}")
            # ToolMessage -> result of a tool call
            elif m.__class__.__name__ == "ToolMessage":
                obs = str(m.content)
                if len(obs) > 400:
                    obs = obs[:400] + "... <truncated>"
                print(f"[OBSERVATION] {obs}")
            # Plain AIMessage with content -> agent's thought or final
            elif m.__class__.__name__ == "AIMessage" and m.content:
                print(f"\n[THOUGHT / FINAL] {m.content}")
        print("\n" + "=" * 70)

    return result
```

Test Cases

> **NOTE:** Please update the **receiver email address** with a valid

> sample email ID to test email delivery in this case study. In a real-world system, the employee's email address would be fetched directly from the database and automatically populated here.

In [ ]:
```
receiver_email = "omm7official@gmail.com"
```

Through all the test cases below, the input to the agent is a single reimbursement request with the following attributes:

- **Emp_ID**: Unique identifier of the employee submitting the reimbursement
- **emp_name**: Name of the employee
- **image**: Upload the image of the bill

These inputs are sent as a single natural-language task to the agent. The agent's system prompt and tool registry handle everything from there.

Test Case 1

In this test case, the employee submits a food reimbursement request that exceeds the available food allowance limit.

Watch the trace: the agent should call `structure_bill`, then `validate_structured_bill`, then `get_reimbursement_caps`, `get_past_utilization`, `calculate_approved_reimbursement`, `generate_decision_email`, and finally `send_decision_email`.

- EMP_ID = E001
- EMP_NAME = Amit Sharma

In [ ]:
```
result_1 = run_reimbursement_agent()
```

```
Enter employee ID: E001
Enter employee name: Amit Sharma
Upload the Bill
```

Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Screenshot 2026-06-29 154234.png to Screenshot 2026-06-29 15423
4.png
INFO:     127.0.0.1:33540 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33556 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33558 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:33568 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33580 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:33582 - "GET /mcp HTTP/1.1" 200 OK
```

```
INFO:     127.0.0.1:33592 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33602 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33616 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33620 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33634 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33636 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:44552 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:44556 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:44568 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:44570 - "POST /mcp HTTP/1.1" 200 OK
[INSIDE TOOL]: send_email to=omm7official@gmail.com
INFO:     127.0.0.1:44578 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:44584 - "DELETE /mcp HTTP/1.1" 200 OK
======================================================================
AGENT TRACE
======================================================================

[ACTION] extract_bill_from_image
[ACTION INPUT]
{
  "image": "/content/Screenshot 2026-06-29 154234.png"
}
[OBSERVATION] "THE OAK STREET BISTRO
245 Oak Street, San Francisco, CA 94102
Phone No: (415) 555-2389
Liquor License No: CA-LL-558912
Sales Tax ID: CA-ST-784512

RESTAURANT
Bill No: SF1983
Date: 14/01/26
Time: 19:45
Table: 12
Cover: 2
Server: Emily
POS ID: eml_07

Item Details

Item Name          Qty     Rate (USD)     Amount
Liquor - Classic Margarita     2     14.00     28.00
L... <truncated>

[ACTION] structure_bill
[ACTION INPUT]
{
  "raw_bill": "THE OAK STREET BISTRO \n245 Oak Street, San Francisco, C
A 94102 \nPhone No: (415) 555-2389 \nLiquor License No: CA-LL-558912 \n
Sales Tax ID: CA-ST-784512 \n\nRESTAURANT \nBill No: SF1983 \nDate: 14/
01/26 \nTime: 19:45 \nTable: 12 \nCover: 2 \nServer: Emily \nPOS ID: em
l_07 \n\nItem Details  \n\nItem Name          Qty     Rate (USD)     Am
ount  \nLiquor - Classic Margarita     2   ... <truncated>
```

[OBSERVATION] {
  "date": "2026-01-14",
  "vendor_details": "THE OAK STREET BISTRO | 245 Oak Street, San Franci
sco, CA 94102 | Invoice #SF1983",
  "food_bill_amount": 110.13,
  "travel_bill_amount": 0,
  "hotel_bill_amount": 0,
  "others_bill_amount": 0,
  "total_bill_amount": 110.13
}

[ACTION] validate_structured_bill
[ACTION INPUT]
{
  "raw_bill": "THE OAK STREET BISTRO \n245 Oak Street, San Francisco, C
A 94102 \nPhone No: (415) 555-2389 \nLiquor License No: CA-LL-558912 \n
Sales Tax ID: CA-ST-784512 \n\nRESTAURANT \nBill No: SF1983 \nDate: 14/
01/26 \nTime: 19:45 \nTable: 12 \nCover: 2 \nServer: Emily \nPOS ID: em
l_07 \n\nItem Details \n\nItem Name   Qty   Rate (USD)   Am
ount \nLiquor - Classic Margarita   2   ... <truncated>
[OBSERVATION] {
  "decision": "APPROVE",
  "rationale": "The total food bill amount of 110.13 is deterministical
ly derived from the sum of base charges (93.00) and applicable taxes (L
iquor Tax 2.76, Service Charge 7.44, Sales Tax 6.93) from the raw bill.
All category totals are tax-inclusive and match the structured data exa
ctly."
}

[ACTION] get_reimbursement_caps
[ACTION INPUT]
{
  "emp_id": "E001"
}

[ACTION] get_past_utilization
[ACTION INPUT]
{
  "emp_id": "E001"
}
[OBSERVATION] {"food": 100.0, "travel": 200.0, "hotel": 300.0}
[OBSERVATION] {"food": 94.07, "travel": 60.7, "hotel": 85.09}

[ACTION] calculate_approved_reimbursement
[ACTION INPUT]
{
  "food_bill_amount": 110.13,
  "travel_bill_amount": 0,
  "hotel_bill_amount": 0,
  "food_cap": 100.0,
  "travel_cap": 200.0,
  "hotel_cap": 300.0,

```
  "food_used": 94.07,
  "travel_used": 60.7,
  "hotel_used": 85.09
}
[OBSERVATION] {"food": {"requested_amount": 110.13, "remaining_cap": 5.
930000000000007, "approved_amount": 5.930000000000007}, "travel": {"req
uested_amount": 0.0, "remaining_cap": 139.3, "approved_amount": 0.0}, "
hotel": {"requested_amount": 0.0, "remaining_cap": 214.91, "approved_am
ount": 0.0}}

[ACTION] generate_decision_email
[ACTION INPUT]
{
  "emp_id": "E001",
  "emp_name": "Amit Sharma",
  "transaction_date": "2026-01-14",
  "vendor_details": "THE OAK STREET BISTRO | 245 Oak Street, San Franci
sco, CA 94102 | Invoice #SF1983",
  "calculation_summary_json": "{\"food\": {\"requested_amount\": 110.1
3, \"remaining_cap\": 5.930000000000007, \"approved_amount\": 5.9300000
00000007}, \"travel\": {\"requested_amount\": 0.0, \"remaining_cap\...
<truncated>
[OBSERVATION] {
  "subject": "Reimbursement Decision for Transaction on 2026-01-14",
  "mail_text": "Dear Amit Sharma,\n\nWe are writing to inform you of th
e reimbursement decision regarding your transaction dated 2026-01-14 wi
th THE OAK STREET BISTRO, located at 245 Oak Street, San Francisco, CA
94102, Invoice #SF1983.\n\nThe requested amount for the Food category w
as $110.13. However, the remaining cap for th... <truncated>

[ACTION] send_decision_email
[ACTION INPUT]
{
  "receiver_email": "omm7official@gmail.com",
  "subject": "Reimbursement Decision for Transaction on 2026-01-14",
  "mail_text": "Dear Amit Sharma,\n\nWe are writing to inform you of th
e reimbursement decision regarding your transaction dated 2026-01-14 wi
th THE OAK STREET BISTRO, located at 245 Oak Street, San Francisco, CA
94102, Invoice #SF1983.\n\nThe requested amount for the Food category
... <truncated>
[OBSERVATION] {"status": "sent"}

[THOUGHT / FINAL] ### Summary of Reimbursement Request:

- **Food Category:**
  - Requested Amount: $110.13
  - Approved Amount: $5.93 (due to cap limitations)

- **Travel Category:**
  - Requested Amount: $0.00
  - Approved Amount: $0.00
```

```
- **Hotel Category:**
  - Requested Amount: $0.00
  - Approved Amount: $0.00

- **Total Approved Amount:** $5.93

- **Email Status:** Sent successfully to omm7official@gmail.com

The reimbursement was limited by the remaining cap in the food categor
y, resulting in a partial approval.

======================================================================
```

Observation: The email communicates the reimbursement decision clearly and professionally, providing sufficient context and rationale to ensure transparency and employee understanding.

Test Case 2

In this test case, the employee submits a reimbursement request that includes food and liquor items with applicable taxes (service charge and sales tax) and travel expenses, and the total amount exceeds the remaining eligible food cap.

- EMP_ID = E002
- EMP_NAME = Neha Verma

In [ ]:
```
result_2 = run_reimbursement_agent()
```

```
Enter employee ID: E002
Enter employee name: Neha Verma
Upload the Bill
```

Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving tc90.png to tc90.png
INFO:     127.0.0.1:52194 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52192 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52210 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:52218 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52228 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:52238 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52240 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52254 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52262 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52272 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52278 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:52284 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:60014 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:60028 - "POST /mcp HTTP/1.1" 202 Accepted
```

```
INFO:     127.0.0.1:60044 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:60052 - "POST /mcp HTTP/1.1" 200 OK
[INSIDE TOOL]: send_email to=omm7official@gmail.com
INFO:     127.0.0.1:60054 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:60066 - "DELETE /mcp HTTP/1.1" 200 OK
================================================================================
AGENT TRACE
================================================================================

[ACTION] extract_bill_from_image
[ACTION INPUT]
{
  "image": "/content/tc90.png"
}
[OBSERVATION] "THE OAK STREET BISTRO
245 Oak Street, San Francisco, CA 94102
Phone No: (415) 555-2389
Liquor License No: CA-LL-558912
Sales Tax ID: CA-ST-784512

RESTAURANT
Bill No: SF10983
Date: 14/01/26
Time: 19:45
Table: 12
Cover: 2
Server: Emily
POS ID: em1_07

FOOD & BEVERAGE DETAILS
Item Name                          Qty     Rate (USD)     Amount
------------------------------... <truncated>

[ACTION] structure_bill
[ACTION INPUT]
{
  "raw_bill": "THE OAK STREET BISTRO \n245 Oak Street, San Francisco, C
A 94102 \nPhone No: (415) 555-2389 \nLiquor License No: CA-LL-558912 \n
Sales Tax ID: CA-ST-784512 \n\nRESTAURANT \nBill No: SF10983 \nDate: 1
4/01/26 \nTime: 19:45 \nTable: 12 \nCover: 2 \nServer: Emily \nPOS ID:
em1_07 \n\nFOOD & BEVERAGE DETAILS \nItem Name Qty Rate (USD) Amount \n
------------------------------------------... <truncated>
[OBSERVATION] {
  "date": "2026-01-14",
  "vendor_details": "THE OAK STREET BISTRO | 245 Oak Street, San Franci
sco, CA 94102 | Invoice #SF10983",
  "food_bill_amount": 110.13,
  "travel_bill_amount": 149.00,
  "hotel_bill_amount": 0,
  "others_bill_amount": 0,
  "total_bill_amount": 259.13
}
```

[ACTION] validate_structured_bill
[ACTION INPUT]
{
  "raw_bill": "THE OAK STREET BISTRO \n245 Oak Street, San Francisco, C
A 94102 \nPhone No: (415) 555-2389 \nLiquor License No: CA-LL-558912 \n
Sales Tax ID: CA-ST-784512 \n\nRESTAURANT \nBill No: SF10983 \nDate: 1
4/01/26 \nTime: 19:45 \nTable: 12 \nCover: 2 \nServer: Emily \nPOS ID:
em1_07 \n\nFOOD & BEVERAGE DETAILS \nItem Name Qty Rate (USD) Amount \n
----------... [OBSERVATION] {
  "decision": "APPROVE",
  "rationale": "Totals are deterministically derived and tax-inclusive.
The food and beverage total of 110.13 includes the base charges of 93.0
0 plus liquor tax of 2.76, service charge of 7.44, and sales tax of 6.9
3. The travel total of 149.00 matches the structured data. The overall
total of 259.13 is the sum of the food and beverage total and the trave
l total, with no ...

[ACTION] get_reimbursement_caps
[ACTION INPUT]
{
  "emp_id": "E002"
}

[ACTION] get_past_utilization
[ACTION INPUT]
{
  "emp_id": "E002"
}
[OBSERVATION] {"food": 150.0, "travel": 250.0, "hotel": 350.0}
[OBSERVATION] {"food": 95.61999999999999, "travel": 536.75, "hotel": 24
3.81}

[ACTION] calculate_approved_reimbursement
[ACTION INPUT]
{
  "food_bill_amount": 110.13,
  "travel_bill_amount": 149.0,
  "hotel_bill_amount": 0,
  "food_cap": 150.0,
  "travel_cap": 250.0,
  "hotel_cap": 350.0,
  "food_used": 95.61999999999999,
  "travel_used": 536.75,
  "hotel_used": 243.81
}
[OBSERVATION] {"food": {"requested_amount": 110.13, "remaining_cap": 5
4.38000000000001, "approved_amount": 54.38000000000001}, "travel": {"re
quested_amount": 149.0, "remaining_cap": 0.0, "approved_amount": 0.0},
"hotel": {"requested_amount": 0.0, "remaining_cap": 106.19, "approved_a
mount": 0.0}}

```
[ACTION] generate_decision_email
[ACTION INPUT]
{
  "emp_id": "E002",
  "emp_name": "Neha Verma",
  "transaction_date": "2026-01-14",
  "vendor_details": "THE OAK STREET BISTRO | 245 Oak Street, San Franci
sco, CA 94102 | Invoice #SF10983",
  "calculation_summary_json": "{\"food\": {\"requested_amount\": 110.1
3, \"remaining_cap\": 54.38000000000001, \"approved_amount\": 54.380000
00000001}, \"travel\": {\"requested_amount\": 149.0, \"remaining_ca...
<truncated>
[OBSERVATION] {
  "subject": "Reimbursement Decision for Transaction on 2026-01-14",
  "mail_text": "Dear Neha Verma,\n\nWe are writing to inform you of the
reimbursement decision regarding your recent expense submission dated 2
026-01-14, associated with THE OAK STREET BISTRO, located at 245 Oak St
reet, San Francisco, CA 94102, Invoice #SF10983.\n\nAfter reviewing the
submitted expenses, the following decisions... <truncated>

[ACTION] send_decision_email
[ACTION INPUT]
{
  "receiver_email": "omm7official@gmail.com",
  "subject": "Reimbursement Decision for Transaction on 2026-01-14",
  "mail_text": "Dear Neha Verma,\n\nWe are writing to inform you of the
reimbursement decision regarding your recent expense submission dated 2
026-01-14, associated with THE OAK STREET BISTRO, located at 245 Oak St
reet, San Francisco, CA 94102, Invoice #SF10983.\n\nAfter reviewing
t... <truncated>
[OBSERVATION] {"status": "sent"}

[THOUGHT / FINAL] ### Summary of Reimbursement Request

- **Food Category:**
  - Requested Amount: $110.13
  - Approved Amount: $54.38 (limited by remaining cap)

- **Travel Category:**
  - Requested Amount: $149.00
  - Approved Amount: $0.00 (no remaining cap)

- **Hotel Category:**
  - Requested Amount: $0.00
  - Approved Amount: $0.00

- **Others Category:**
  - Requested Amount: $0.00
  - Approved Amount: $0.00

- **Total Approved Amount:** $54.38
```

```
- **Email Status:** Sent successfully to Neha Verma at omm7official@gma
il.com.

======================================================================
```

Observation: The email conveys the reimbursement outcome clearly and professionally, with a transparent explanation that reinforces policy compliance while maintaining a courteous and employee-friendly tone.

Test Case 3

In this test case, the employee submits a reimbursement request covering three categories: food, hotel, travel and others, where the total eligible amount exceeds the remaining allowed limit.

- EMP_ID = E003
- EMP_NAME = Rahul Mehta

In [ ]:
```
result_3 = run_reimbursement_agent()
```

```
Enter employee ID: E003
Enter employee name: Rahul Mehta
Upload the Bill
```

Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving tc3.png to tc3.png
INFO:     127.0.0.1:49780 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49764 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49784 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:49788 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49796 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49798 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:49804 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49812 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49822 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49842 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49830 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49844 - "DELETE /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:54822 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:54838 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:54840 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:54854 - "POST /mcp HTTP/1.1" 200 OK
[INSIDE TOOL]: send_email to=omm7official@gmail.com
INFO:     127.0.0.1:54860 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:54872 - "DELETE /mcp HTTP/1.1" 200 OK
======================================================================
AGENT TRACE
```

```
======================================================================

[ACTION] extract_bill_from_image
[ACTION INPUT]
{
  "image": "/content/tc3.png"
}
[OBSERVATION] EXPENSE SUMMARY BY CATEGORY

Category                       Description
Total Amount (USD)
Food                            Meals during business trip
250.00
Hotel                           Accommodation expenses
350.00
Travel                          Travel expenses during business trip
652.60
Others      ... <truncated>

[ACTION] structure_bill
[ACTION INPUT]
{
  "raw_bill": "EXPENSE SUMMARY BY CATEGORY\n\nCategory
Description                                          Total Amount (USD)\nFo
od                            Meals during business trip
250.00\nHotel                             Accommodation expenses
350.00\nTravel                            Travel expenses during business
trip          ... <truncated>
[OBSERVATION] {
  "date": "2026-01-15",
  "vendor_details": null,
  "food_bill_amount": 250.00,
  "travel_bill_amount": 652.60,
  "hotel_bill_amount": 350.00,
  "others_bill_amount": 52.60,
  "total_bill_amount": 1305.20
}

[ACTION] validate_structured_bill
[ACTION INPUT]
{
  "raw_bill": "EXPENSE SUMMARY BY CATEGORY\n\nCategory
Description                                          Total Amount (USD)\nFo
od                            Meals during business trip
250.00\nHotel                             Accommodation expenses
350.00\nTravel                            Travel expenses during business
trip          ... <truncated>
[OBSERVATION] {
  "decision": "APPROVE",
  "rationale": "All category totals are deterministically derived and t
ax-inclusive. The structured data matches the recomposed totals from th
e raw bill text exactly."
```

```
}

[ACTION] get_reimbursement_caps
[ACTION INPUT]
{
  "emp_id": "E003"
}

[ACTION] get_past_utilization
[ACTION INPUT]
{
  "emp_id": "E003"
}
[OBSERVATION] {"food": 200.0, "travel": 300.0, "hotel": 400.0}
[OBSERVATION] {"food": 79.38, "travel": 223.57999999999998, "hotel": 43
4.22}

[ACTION] calculate_approved_reimbursement
[ACTION INPUT]
{
  "food_bill_amount": 250.0,
  "travel_bill_amount": 652.6,
  "hotel_bill_amount": 350.0,
  "food_cap": 200.0,
  "travel_cap": 300.0,
  "hotel_cap": 400.0,
  "food_used": 79.38,
  "travel_used": 223.57999999999998,
  "hotel_used": 434.22
}
[OBSERVATION] {"food": {"requested_amount": 250.0, "remaining_cap": 12
0.62, "approved_amount": 120.62}, "travel": {"requested_amount": 652.6,
"remaining_cap": 76.42000000000002, "approved_amount": 76.4200000000000
2}, "hotel": {"requested_amount": 350.0, "remaining_cap": 0.0, "approve
d_amount": 0.0}}

[ACTION] generate_decision_email
[ACTION INPUT]
{
  "emp_id": "E003",
  "emp_name": "Rahul Mehta",
  "transaction_date": "2026-01-15",
  "vendor_details": null,
  "calculation_summary_json": "{\"food\": {\"requested_amount\": 250.0,
\"remaining_cap\": 120.62, \"approved_amount\": 120.62}, \"travel\": {
\"requested_amount\": 652.6, \"remaining_cap\": 76.42000000000002, \"ap
proved_amount\": 76.42000000000002}, \"hotel\": {\"requested_amount\":
350... <truncated>
[OBSERVATION] Error invoking tool 'generate_decision_email' with kwargs
{'emp_id': 'E003', 'emp_name': 'Rahul Mehta', 'transaction_date': '2026
-01-15', 'vendor_details': None, 'calculation_summary_json': '{"food":
{"requested_amount": 250.0, "remaining_cap": 120.62, "approved_amount":
```

```
120.62}, "travel": {"requested_amount": 652.6, "remaining_cap": 76.4200
0000000002, "approved_amount": 76.42000000000002}, "hotel... <truncated
>

[ACTION] generate_decision_email
[ACTION INPUT]
{
  "emp_id": "E003",
  "emp_name": "Rahul Mehta",
  "transaction_date": "2026-01-15",
  "vendor_details": "N/A",
  "calculation_summary_json": "{\"food\": {\"requested_amount\": 250.0,
\"remaining_cap\": 120.62, \"approved_amount\": 120.62}, \"travel\": {
\"requested_amount\": 652.6, \"remaining_cap\": 76.42000000000002, \"ap
proved_amount\": 76.42000000000002}, \"hotel\": {\"requested_amount\":
35... <truncated>
[OBSERVATION] {
  "subject": "Reimbursement Decision for Transaction on 2026-01-15",
  "mail_text": "Dear Rahul Mehta,\n\nWe have reviewed your reimbursemen
t request submitted for the transaction dated 2026-01-15. Please find t
he details of the reimbursement decision below:\n\n**Food:**\nThe reque
sted amount for food was $250.00. However, the remaining cap for this c
ategory was $120.62. Therefore, the approved ... <truncated>

[ACTION] send_decision_email
[ACTION INPUT]
{
  "receiver_email": "omm7official@gmail.com",
  "subject": "Reimbursement Decision for Transaction on 2026-01-15",
  "mail_text": "Dear Rahul Mehta,\n\nWe have reviewed your reimbursemen
t request submitted for the transaction dated 2026-01-15. Please find t
he details of the reimbursement decision below:\n\n**Food:**\nThe reque
sted amount for food was $250.00. However, the remaining cap for this
... <truncated>
[OBSERVATION] {"status": "sent"}

[THOUGHT / FINAL] ### Summary of Reimbursement Request:

- **Food:**
  - Requested: $250.00
  - Approved: $120.62

- **Travel:**
  - Requested: $652.60
  - Approved: $76.42

- **Hotel:**
  - Requested: $350.00
  - Approved: $0.00

- **Others:**
  - Non-reimbursable: $52.60
```

```
- **Total Bill Amount:** $1305.20
- **Total Approved Amount:** $197.04

### Email Status:
- The reimbursement decision email was successfully sent to Rahul Mehta
at omm7official@gmail.com.

==========================================================================
```

Observation: The agent provides a clear, well-structured breakdown of category-wise reimbursement decisions, transparently explaining both approved and non-approved amounts while maintaining a professional tone.

Conclusions and Business Recommendations

- **Use agentic reasoning for dynamic decision-making:** Agent-based workflows are well-suited for business processes where the next action depends on the outcome of previous steps, enabling adaptive and context-aware execution.
- **Integrate external tools through controlled interfaces:** Connecting agents to enterprise systems, databases, and communication channels through governed tool access ensures secure and reliable execution of business actions.
- **Capture business procedures as configurable instructions:** Defining operational guidelines and decision criteria in prompts or configuration layers enables faster updates and reduces dependence on code changes for process improvements.
- **Maintain transparency through execution traces:** Recording the agent's reasoning, tool usage, and intermediate observations provides a clear audit trail that supports monitoring, troubleshooting, and compliance requirements.
- **Balance flexibility with governance:** Combining autonomous decision-making with validation checks, business rules, and approval mechanisms helps ensure that outcomes remain accurate, compliant, and aligned with organizational policies.